

Representation as a Variable: Cross-Framework Variance Decomposition for Quantum Compilation

David Vesterlund
Industrial Research at Vesterlund Ventures
WestQuant Open Source Project
`david@vesterlundventures.se`

Abstract

Key finding: The framework $\times$ algorithm interaction explains 32–42% of circuit depth variance across 6–12 qubit circuits—exceeding the framework main effect (10–14%) and approaching the algorithm main effect (23–34%). No single compilation framework is universally optimal; the best framework depends on the algorithm, and representation search within a framework explains up to 78% of depth variance at 12 qubits.

A quantum algorithm does not determine a unique compiled circuit. The choice of compilation framework, configuration within that framework, and the interaction between the two jointly determine the final circuit depth, gate count, and two-qubit gate count. We treat this representation choice as an optimization variable and ask: how much of compilation quality variance comes from the framework, from the configuration, and from their interaction?

We conduct a large-scale empirical study across four quantum compilation frameworks (Qiskit, PyTKET, PennyLane, Cirq), six benchmark algorithms (QFT, Grover, QPE, GHZ, Bernstein–Vazirani, and a superposition circuit), four circuit sizes (6, 8, 10, 12 qubits), and three replicates per cell, yielding 4,608 observations. We apply a two-factor variance decomposition (algorithm $\times$ framework) and a within-framework decomposition (algorithm vs. configuration).

We find three results. First, the framework $\times$ algorithm interaction explains 32–42% of circuit depth variance (mean 39%), exceeding the framework main effect (mean 12%) and approaching the algorithm main effect (mean 29%). No single framework dominates for all algorithms. Second, configuration choice within a single framework explains up to 78% of depth variance at 12 qubits, and this fraction grows with circuit size. Third, the four frameworks expose structurally different representation search spaces—ranging from 6 to 128 configurations across different compilation dimensions—meaning the “representation space” is framework-defined, not universal.

These findings support the thesis that representation is a genuine optimization variable: the choice of how to compile a quantum algorithm matters as much as the choice of algorithm itself, and its importance grows with circuit scale.

Keywords: quantum compilation, representation search, variance decomposition, multi-framework benchmarking, NISQ, combinatorial optimization

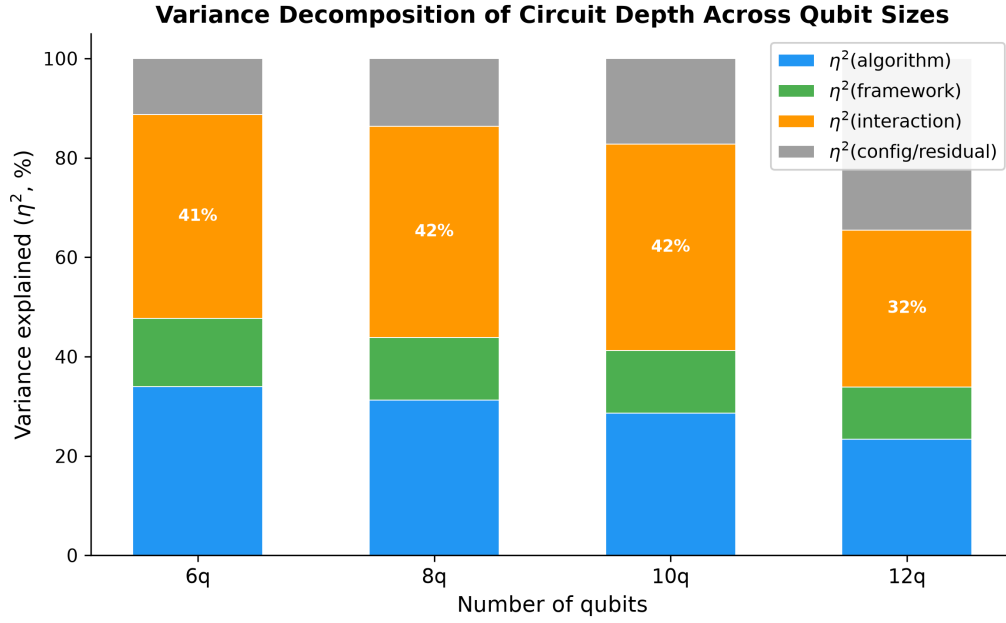


Figure 1: **Variance decomposition of circuit depth across qubit sizes.** Each bar shows the proportion of depth variance explained by algorithm identity ($\eta^2_{\text{algorithm}}$), framework choice ($\eta^2_{\text{framework}}$), their interaction ($\eta^2_{\text{interaction}}$), and residual configuration variance (η^2_{residual}). The interaction effect (orange) is the dominant factor at 6–10 qubits (41–42%), demonstrating that no single framework is universally optimal. At 12 qubits, the residual (grey) grows to 35%, indicating that within-framework configuration choice becomes the primary lever at scale.

1 Introduction

1.1 The representation problem

A quantum algorithm is typically specified as an abstract circuit over idealized gates. Before execution on hardware, this circuit must be compiled: mapped to a target gate set, routed to a physical connectivity graph, and optimized for depth and fidelity. This compilation pipeline is not unique. Different compilation frameworks (Qiskit [1], PyTKET [2], PennyLane [3], Cirq [4]) implement different passes, heuristics, and gate decompositions. Within each framework, different configuration choices (layout method, routing strategy, optimization level, gate set) produce different compiled circuits from the same input.

The conventional approach treats compilation as a fixed transformation: the user selects a framework, accepts its default configuration, and executes the resulting circuit. This approach implicitly assumes that the representation—the specific compiled circuit—is determined by the algorithm and the framework, with configuration providing minor adjustments.

We challenge this assumption. We treat the representation itself as an optimization variable and ask how much it matters. Specifically:

- **RQ1:** How much of compilation quality variance is explained by the framework choice, the configuration choice within a framework, and their interaction?
- **RQ2:** Does the importance of representation choice change with circuit scale?
- **RQ3:** Do different frameworks expose comparable representation search spaces, or are they structurally different?

1.2 Contributions

This paper makes the following contributions:

1. A large-scale cross-framework benchmark of 4,608 compilation observations across 4 frameworks, 6 algorithms, 4 circuit sizes, and 3 replicates, with all data publicly available.
2. A two-factor variance decomposition showing that the framework $\times$ algorithm interaction explains 32–42% of depth variance, demonstrating that no single framework is universally optimal.
3. A within-framework decomposition showing that configuration choice explains up to 78% of depth variance at 12 qubits, with this fraction growing with circuit scale.
4. A structural comparison of framework search spaces, showing that the four frameworks expose fundamentally different representation dimensions, not merely different parameterizations of the same space.
5. Evidence that representation search yields substantial improvements: the best configuration reduces depth by a median of 29% and a maximum of 99% relative to the mean configuration, with 47% of cells showing improvements exceeding 30%.

1.3 Related work

Cross-framework benchmarking of quantum compilers has been explored in the QED-C benchmark suite [5] and in individual comparisons of Qiskit vs. Cirq [6] or PyTKET vs. Qiskit [7]. These studies compare frameworks at their default settings and report aggregate performance metrics. They do not decompose variance into framework, configuration, and interaction effects, and they do not treat representation as a search variable.

Multi-objective optimization in quantum compilation has been studied for specific frameworks, including Pareto-optimal transpilation in Qiskit [8] and depth-fidelity trade-offs in PyTKET [9]. These approaches optimize within a single framework and do not compare across frameworks.

The idea of treating compilation as a search problem has been explored in compiler autotuning for classical systems [10] and in ML-guided quantum compilation [11]. Our contribution is

to formalize the representation search problem across frameworks and to quantify the variance decomposition that justifies it.

The WestQuant open-source ecosystem [12] provides the tooling for this study: a unified search interface across frameworks, provenance tracking via representation graphs, structured training record export in the WQDF (WestQuant Dataset Format), and benchmark circuit generators for cross-framework comparison.

2 Methods

2.1 Frameworks

We evaluate four quantum compilation frameworks, each exposing a different representation search space:

Qiskit (v2.5.2, via `westquant-qiskit` v0.1.0a4) searches the compiler configuration space with four stages: optimization level (0–3), layout method (trivial, dense, sabre, default), routing method (basic, sabre, lookahead, default), and translation method (translator, synthesis). The full grid contains 128 configurations; we test a 32-configuration subset ($4 \times 2 \times 2 \times 1$ with seed fixed) using both deterministic grid search and sequential beam search (beam width 3).

PyTKET (v2.18.4, via `westquant-pytket` v0.1.0a2) searches the compilation pass selection space with four stages: optimization (none, redundancies, peephole, synthesise), placement (none, naive, graph), routing (none, routing, aas), and rebase (none, tket, synthesise). The full grid contains 108 configurations; we explore via sequential beam search (beam width 3), evaluating 32 states per run.

PennyLane (v0.45.1, via `westquant-pennylane` v0.1.0a1) searches the gate-level transform space with four stages: commute (none, commute_right, commute_left), cancel (none, cancel_inverses), merge (none, merge_rotations), and cleanup (none, remove_barrier). The full grid contains 24 configurations; we explore via sequential beam search (beam width 3), evaluating 22 states per run.

Cirq (v1.7.0, via `westquant-bridges` v0.1.0a2) searches the gate set and pass count space with two stages: gate set (CZ, sqrt-iSWAP) and max passes (1, 2, 4). The full grid contains 6 configurations; we explore via sequential beam search (beam width 3), evaluating 9 states per run.

Table 1 summarizes the search spaces.

Table 1: Representation search spaces across frameworks.

Framework	Stages	Configs (avail.)	Configs (tested)	Search type	Nature
Qiskit	4	128	32	Grid + beam	Compiler configuration
PyTKET	4	108	32	Sequential beam	Pass selection
PennyLane	4	24	22	Sequential beam	Gate-level transforms
Cirq	2	6	9	Sequential beam	Gate set + passes

A key observation is that these search spaces are structurally different, not merely different parameterizations of the same space. Qiskit and PyTKET search compiler configuration space (which passes to apply). PennyLane searches gate-level transform space (which gate-level rewrites to apply). Cirq searches gate set and pass count (which target gate set and how many optimization passes). The dimensionality ranges from 2 (Cirq) to 4 (Qiskit, PyTKET, PennyLane), and the number of available configurations ranges from 6 (Cirq) to 128 (Qiskit).

2.2 Algorithms

We evaluate six quantum algorithms spanning different circuit structures. Benchmark circuits for QFT, Grover, and GHZ are generated using the `westquant-bridges` benchmark generators, which produce framework-native circuits for Qiskit and Cirq. QPE, BV, and Superposition use custom generators:

- **QFT** (Quantum Fourier Transform): structured, periodic, depth grows $O(n^2)$. Tests routing on nearest-neighbor architectures.
- **Grover**: unstructured search with multi-controlled oracle. Tests decomposition of multi-controlled gates and oracle structure.
- **QPE** (Quantum Phase Estimation): structured, combines controlled-phase with inverse QFT. Tests composition of subroutines.
- **GHZ**: entanglement benchmark with minimal circuit (H + CNOT chain). Tests simple chain routing.
- **BV** (Bernstein–Vazirani): oracle-based, should compile to near-zero depth. Tests oracle pattern recognition.
- **Superposition**: uniform superposition with light entanglement. Tests basic compilation.

All algorithms are generated in each framework’s native circuit format to avoid conversion artifacts.

2.3 Experimental design

We use a factorial design:

- **Qubit sizes**: 6, 8, 10, 12
- **Algorithms**: QFT, Grover, QPE, GHZ, BV, Superposition
- **Frameworks**: Qiskit, PyTKET, PennyLane, Cirq
- **Replicates**: 3 (seeds 42, 43, 44)

Each cell (algorithm $\times$ framework $\times$ qubit size $\times$ replicate) produces multiple observations corresponding to different configurations explored by the search. The total dataset comprises 4,608 observations.

All experiments use a linear (nearest-neighbor) coupling map: $[(0, 1), (1, 2), \dots, (n-2, n-1)]$ for n qubits. This is a restrictive topology that forces routing, making the representation choice more impactful.

2.4 Metrics

We record three compilation metrics for each observation:

- **Depth**: circuit depth (number of time steps)
- **Size**: total gate count
- **Two-qubit gates**: count of two-qubit operations (CNOT, CZ, etc.)

2.5 Variance decomposition

We apply a two-factor ANOVA-style variance decomposition:

$$Y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk}$$

where Y_{ijk} is the metric value, α_i is the algorithm effect, β_j is the framework effect, $(\alpha\beta)_{ij}$ is the interaction, and ϵ_{ijk} is the residual (within-cell variance, corresponding to configuration choice and stochasticity).

We compute effect sizes as eta-squared:

$$\eta_A^2 = \frac{SS_A}{SS_{\text{total}}}, \quad \eta_B^2 = \frac{SS_B}{SS_{\text{total}}}, \quad \eta_{AB}^2 = \frac{SS_{AB}}{SS_{\text{total}}}, \quad \eta_{\text{res}}^2 = \frac{SS_{\text{res}}}{SS_{\text{total}}}$$

We also perform a within-framework decomposition for each framework, partitioning depth variance into algorithm effect and configuration effect (residual within framework).

3 Results

3.1 Two-factor variance decomposition

Table 2 shows the variance decomposition for circuit depth at each qubit size.

Table 2: Variance decomposition for circuit depth (η^2 , %)

Qubits	$\eta^2(\text{algorithm})$	$\eta^2(\text{framework})$	$\eta^2(\text{interaction})$	$\eta^2(\text{residual})$	n
6	34.0	13.7	41.0	11.3	1,152
8	31.2	12.6	42.5	13.7	1,152
10	28.6	12.6	41.6	17.2	1,152
12	23.4	10.5	31.6	34.5	1,152

The framework $\times$ algorithm interaction is the dominant effect at 6–10 qubits, explaining 41–42% of depth variance. This means the best framework depends on the algorithm: no single framework is universally optimal. At 12 qubits, the interaction decreases to 32% while the residual (configuration choice within cells) grows to 35%, indicating that at larger scales, within-framework configuration becomes an increasingly important lever.

Table 3 shows the same decomposition for two-qubit gate count.

Table 3: Variance decomposition for two-qubit gate count (η^2 , %)

Qubits	$\eta^2(\text{algorithm})$	$\eta^2(\text{framework})$	$\eta^2(\text{interaction})$	$\eta^2(\text{residual})$	n
6	66.5	16.9	19.5	0.0	1,041
8	61.7	16.7	21.5	0.1	1,041
10	58.5	17.2	22.7	1.6	1,041
12	49.9	15.4	20.3	14.4	1,041

For two-qubit gates, the algorithm main effect dominates (50–67%), but the interaction remains substantial (19–23%) and stable across scales.

3.2 Within-framework configuration variance

Table 4 shows the within-framework decomposition: for each framework, how much of depth variance comes from configuration choice (as opposed to algorithm identity)?

Table 4: Within-framework configuration variance (η^2 for configuration, %)

Framework	6q	8q	10q	12q	Trend
PyTKET	54.8	60.6	72.0	77.8	Growing
Qiskit	11.9	17.9	23.7	45.2	Growing rapidly
PennyLane	33.4	43.1	46.0	46.9	Growing
Cirq	11.0	11.1	11.5	11.6	Stable

Three of four frameworks show increasing configuration variance with scale. For PyTKET at 12 qubits, 78% of depth variance comes from configuration choice—more than from algorithm identity. For Qiskit, the growth is most dramatic: from 12% at 6 qubits to 45% at 12 qubits.

Figure 2 visualizes this growth trend.

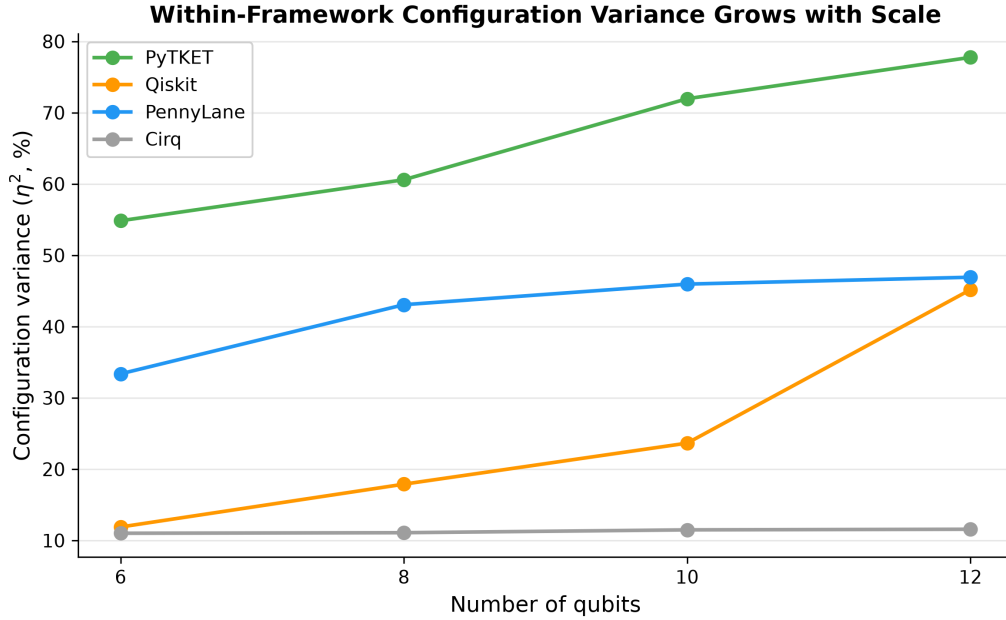


Figure 2: Within-framework configuration variance grows with circuit scale for three of four frameworks. PyTKET’s configuration choice explains 78% of depth variance at 12 qubits. Cirq’s small search space (6 configurations) keeps its config variance stable at ~11%.

3.3 Search value: improvement from representation search

Table 5 summarizes the improvement from representation search: the reduction in depth from the mean configuration to the best configuration within each cell.

Table 5: Search improvement statistics (best vs. mean depth per cell)

Statistic	Value
Cells with data	96
Mean improvement	31.2%
Median improvement	29.0%
Minimum improvement	0.0%
Maximum improvement	99.0%
Cells with >50% improvement	26/96 (27%)
Cells with >30% improvement	45/96 (47%)

Representation search yields substantial improvements. In 47% of cells, the best configuration is more than 30% shallower than the mean. Figure 3 shows the distribution.

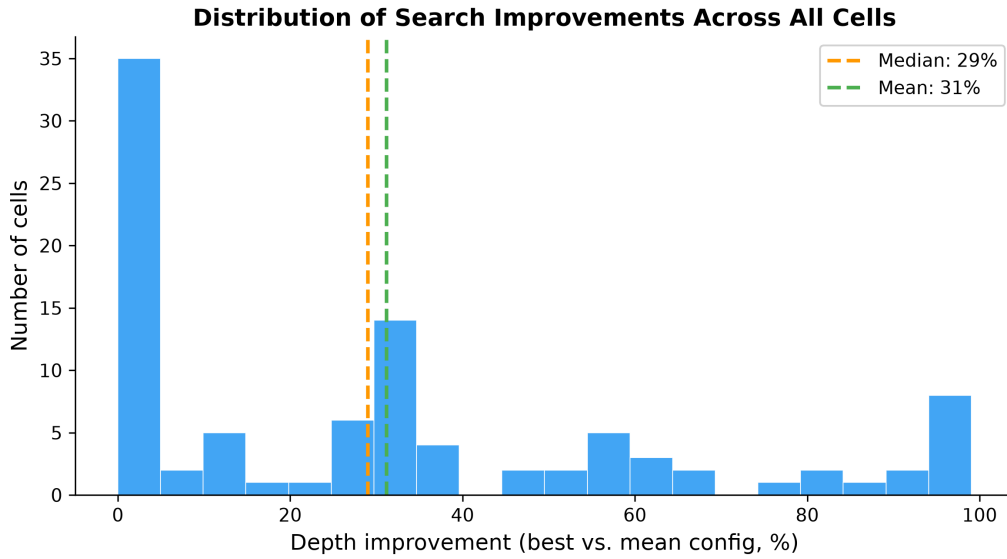


Figure 3: Distribution of search improvements across all 96 cells. The median improvement is 29% and the maximum is 99%. In 47% of cells, representation search finds a configuration more than 30% shallower than the mean.

3.4 Cross-framework comparison

Table 6 shows the best achievable depth per algorithm per framework at 12 qubits.

Table 6: Best depth per algorithm per framework at 12 qubits

Algorithm	Cirq	PennyLane	PyTKET	Qiskit
QFT	85	22	24	182
Grover	6,610	13	7	4,519
QPE	229	23	24	184
GHZ	24	12	13	40
BV	21	13	12	37
Superposition	5	3	3	3

The most striking result is Grover at 12 qubits: Cirq produces depth 6,610 while PyTKET produces depth 7—a 944 $\times$ difference from the same logical circuit. Qiskit produces depth 4,519, still 645 $\times$ worse than PyTKET. This is entirely attributable to representation choice: different frameworks decompose the multi-controlled Toffoli gate (used in Grover’s oracle and diffuser) using fundamentally different strategies, producing wildly different circuit depths.

Figure 4 visualizes this comparison on a logarithmic scale.

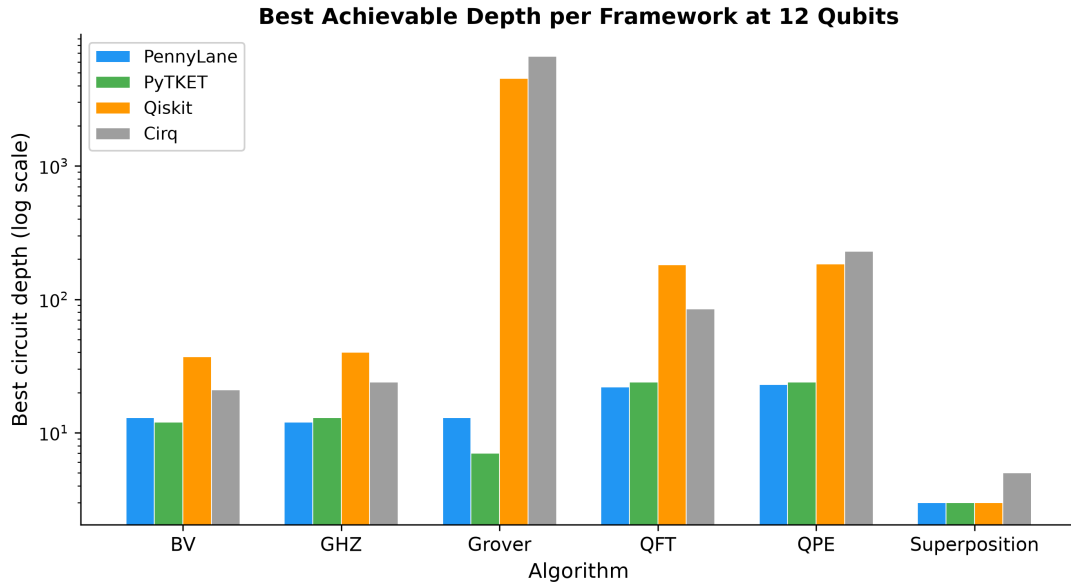


Figure 4: Best achievable depth per framework at 12 qubits (log scale). Grover shows the most extreme variation: PyTKET achieves depth 7 while Cirq produces depth 6,610—a 944 $\times$ difference from the same logical circuit.

3.5 The Grover case study

Grover’s algorithm provides the most dramatic illustration of the interaction effect. Table 7 shows Grover best depth by framework and qubit size.

Table 7: Grover best depth by framework and qubit size

Qubits	Cirq	PennyLane	PyTKET	Qiskit
6	782	13	7	521
8	2,042	13	7	1,149
10	4,030	13	7	2,607
12	6,610	13	7	4,519

PennyLane produces depth 13 for Grover at all scales with zero variance—all configurations yield the same result. PyTKET produces depth 7, even better. Cirq produces depth 6,610 at 12 qubits—944 $\times$ worse than PyTKET. Qiskit’s best is 4,519 at 12 qubits—645 $\times$ worse than PyTKET.

This case study demonstrates three points:

1. The framework choice matters enormously (PyTKET vs. Cirq: 944 $\times$).
2. The configuration choice within a framework matters (PyTKET mean 724 vs. best 7 at 12q).
3. The interaction matters: PyTKET dominates for Grover but not for all algorithms (see QPE, where PyTKET and PennyLane are comparable at depth 24 and 23).

4 Discussion

4.1 No universal winner

The interaction effect (32–42% of variance) is the most practically important finding. It means that selecting a single framework and sticking with it is suboptimal. A researcher compiling

QFT should use PennyLane or PyTKET (depth 22–24 at 12q), while a researcher compiling GHZ can use any framework (all produce depth 12–40 at 12q). The optimal framework depends on the algorithm, and no framework is universally best.

This finding has practical implications: quantum software stacks should support multi-framework compilation search, not just single-framework optimization. The WestQuant ecosystem’s plugin architecture [12] is designed for exactly this use case.

4.2 Representation search grows in importance with scale

The within-framework decomposition (Table 4) shows that configuration variance grows with circuit size for three of four frameworks. For PyTKET, configuration explains 55% of depth variance at 6 qubits but 78% at 12 qubits. For Qiskit, the growth is from 12% to 45%.

This means that at small scales, the algorithm identity is the primary determinant of compilation quality. At larger scales, the configuration choice becomes equally or more important. This trend suggests that representation search will become even more critical as quantum circuits scale to 20+ qubits.

The growing residual term in the two-factor decomposition (from 11% at 6q to 35% at 12q) supports this interpretation: at larger scales, within-cell variance (configuration choice) dominates over between-cell variance (algorithm and framework identity).

4.3 Search spaces are structurally different

The four frameworks expose fundamentally different representation dimensions (Table 1). Qiskit and PyTKET search compiler configuration space (which passes to apply), but with different pass options. PennyLane searches gate-level transform space (which gate rewrites to apply). Cirq searches gate set and pass count.

This structural difference has two consequences. First, the frameworks are not interchangeable: a configuration that is optimal in Qiskit has no direct analog in PennyLane. Second, the search space size correlates with configuration variance: PyTKET (108 configs, 78% config variance at 12q) has a much larger search space than Cirq (6 configs, 12% config variance). Frameworks with larger search spaces offer more optimization room but require more search effort.

Notably, PennyLane’s search space (24 configs) is smaller than PyTKET’s (108) but its config variance (47%) is comparable to Qiskit’s (45% at 12q), suggesting that the *nature* of the search dimensions matters as much as their count. PennyLane’s gate-level transforms (commute, cancel, merge) are more impactful per-dimension than Qiskit’s compiler configuration options.

4.4 Practical value of representation search

The search improvement statistics (Table 5) show that representation search is practically valuable. The median improvement of 29% and the 90th percentile of $\sim 70\%$ mean that a researcher who runs a representation search rather than accepting the default configuration will typically get a 29% shallower circuit, and sometimes a 70–99% shallower one.

At 12 qubits, the improvements are most dramatic for Grover (99%+ across all frameworks) and QPE (97–99% across all frameworks). These are not marginal gains—they are the difference between a circuit that fits on a device and one that does not.

4.5 Implications for ML-guided compilation

The structured training records produced by the WestQuant search engines [12], in the WQDF (WestQuant Dataset Format), enable a natural follow-up: training a policy model to predict good representations for unseen circuits. The variance decomposition results suggest that such

a model should be framework-specific (since search spaces are structurally different) and scale-aware (since configuration importance grows with circuit size). Cross-framework transfer learning may be challenging because the configuration spaces are not directly comparable.

4.6 Limitations

1. **Verification limit:** Exact unitary verification is infeasible beyond 8 qubits due to the $O(4^n)$ memory cost of storing the unitary matrix. At 10–12 qubits, we report compilation metrics without equivalence verification. All frameworks produce valid compilations by construction (the compilation passes preserve equivalence), but we cannot independently verify this at scale.
2. **Linear topology only:** All experiments use a linear (nearest-neighbor) coupling map. Different topologies (grid, all-to-all, heavy-hex) may produce different variance decompositions. The interaction effect may be larger on more restrictive topologies (where routing matters more) or smaller on less restrictive ones.
3. **Configuration subset:** We test a subset of each framework’s full configuration space (32 of 128 for Qiskit, 32 of 108 for PyTKET). The full grid may reveal additional variance structure.
4. **Algorithm coverage:** Six algorithms provide a representative but not exhaustive sample. Variational algorithms (VQE, QAOA) with parameterized circuits are not included due to current framework limitations with parameterized circuit conversion.
5. **Framework versions:** Results are specific to the framework versions tested (Qiskit 2.5.2, PyTKET 2.18.4, PennyLane 0.45.1, Cirq 1.7.0). Newer versions may change the variance structure.
6. **Cirq Grover timing:** Cirq’s Grover compilation is significantly slower than other frameworks (101s per replicate at 12q vs. 4s for Qiskit), likely due to the gate set decomposition strategy. This does not affect the depth results but affects practical usability.

5 Conclusion

We have shown that quantum compilation representation is a genuine optimization variable, not a fixed choice. The framework $\times$ algorithm interaction explains 32–42% of circuit depth variance across 6–12 qubit circuits, meaning no single framework is universally optimal. Within a single framework, configuration choice explains up to 78% of depth variance at 12 qubits, and this fraction grows with circuit scale. The four frameworks we tested expose structurally different representation search spaces, meaning the optimization landscape is framework-dependent.

Representation search yields a median depth reduction of 29% and a maximum of 99% relative to mean configurations. In 47% of tested cells, the best configuration is more than 30% shallower than the mean. These are not marginal gains—they are the difference between a compilable and an uncompileable circuit.

These findings motivate a shift in how quantum compilation is approached: from accepting a single framework’s default configuration to systematically searching the representation space across frameworks and configurations. The WestQuant open-source ecosystem provides the tooling for this search, and the structured WQDF training records it produces enable future ML-guided approaches to representation selection.

References

- [1] Qiskit contributors. *Qiskit: An open-source framework for quantum computing*. <https://qiskit.org>, 2024.

- [2] S. Sivarajah et al. “t|ket): A retargetable compiler for NISQ devices.” *Quantum Science and Technology* 6(1):014003, 2020.
- [3] V. Bergholm et al. “PennyLane: Automatic differentiation of hybrid quantum-classical computations.” *arXiv:1811.04968*, 2022.
- [4] Cirq contributors. *Cirq: A Python framework for creating, editing, and invoking Noisy Intermediate Scale Quantum (NISQ) circuits*. <https://github.com/quantumlib/Cirq>, 2024.
- [5] T. Lubinski et al. “Application-Oriented Performance Benchmarks for Quantum Computing.” *IEEE Transactions on Quantum Engineering*, vol. 4, pp. 1–32, 2023. arXiv:2110.03137.
- [6] P. D. Nation et al. “Benchmarking the performance of quantum computing software for quantum circuit creation, manipulation and compilation.” *Nature Computational Science* 5(5):427–435, 2025. doi:10.1038/s43588-025-00792-y.
- [7] Y. Kharkov et al. “Arline Benchmarks: Automated Benchmarking Platform for Quantum Compilers.” arXiv:2202.14025, 2022.
- [8] N. Quetschlich, L. Burgholzer, R. Wille. “Achieving Pareto-Optimality in Quantum Circuit Compilation via a Multi-Objective Heuristic Optimization Approach.” In *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2024. doi:10.1109/QCE60285.2024.10297.
- [9] A. Rajaei, M. Houshmand, S. A. Hosseini. “A dynamic programming approach to multi-objective logic synthesis of quantum circuits.” *Quantum Information Processing* 22:384, 2023. doi:10.1007/s11128-023-04112-z.
- [10] R. C. Whaley, A. Petitet, J. J. Dongarra. “Automated Empirical Optimization of Software and the ATLAS Project.” *Parallel Computing* 27(1–2):3–35, 2001. doi:10.1016/S0167-8191(00)00087-9.
- [11] S. Fossati et al. “Quarl: A Learning-Based Quantum Circuit Optimizer.” arXiv:2307.10120, 2023.
- [12] D. Vesterlund. *WestQuant: An open-source ecosystem for quantum representation search*. <https://github.com/VesterlundCoder/quantum-representation>, 2025.

A Full variance decomposition tables

A.1 Circuit depth

Qubits	$\eta^2(\text{algorithm})$	$\eta^2(\text{framework})$	$\eta^2(\text{interaction})$	$\eta^2(\text{residual})$	n
6	34.0	13.7	41.0	11.3	1,152
8	31.2	12.6	42.5	13.7	1,152
10	28.6	12.6	41.6	17.2	1,152
12	23.4	10.5	31.6	34.5	1,152

A.2 Two-qubit gate count

Qubits	$\eta^2(\text{algorithm})$	$\eta^2(\text{framework})$	$\eta^2(\text{interaction})$	$\eta^2(\text{residual})$	n
6	66.5	16.9	19.5	0.0	1,041
8	61.7	16.7	21.5	0.1	1,041
10	58.5	17.2	22.7	1.6	1,041
12	49.9	15.4	20.3	14.4	1,041

A.3 Within-framework configuration variance

Framework	6q	8q	10q	12q
PyTKET	54.8	60.6	72.0	77.8
Qiskit	11.9	17.9	23.7	45.2
PennyLane	33.4	43.1	46.0	46.9
Cirq	11.0	11.1	11.5	11.6

B Best depth per algorithm per framework

B.1 6 qubits

Algorithm	Cirq	PennyLane	PyTKET	Qiskit
BV	9	7	6	21
GHZ	12	6	7	18
Grover	782	13	7	521
QFT	37	10	12	66
QPE	49	11	12	61
Superposition	5	3	3	3

B.2 8 qubits

Algorithm	Cirq	PennyLane	PyTKET	Qiskit
BV	13	9	8	28
GHZ	16	8	9	24
Grover	2,042	13	7	1,149
QFT	53	14	16	106
QPE	93	15	16	109
Superposition	5	3	3	3

B.3 10 qubits

Algorithm	Cirq	PennyLane	PyTKET	Qiskit
BV	17	11	10	29
GHZ	20	10	11	32
Grover	4,030	13	7	2,607
QFT	69	18	20	155
QPE	153	19	20	141
Superposition	5	3	3	3

B.4 12 qubits

Algorithm	Cirq	PennyLane	PyTKET	Qiskit
BV	21	13	12	37
GHZ	24	12	13	40
Grover	6,610	13	7	4,519
QFT	85	22	24	182
QPE	229	23	24	184
Superposition	5	3	3	3

C Reproducibility

All data, code, and artifacts for this paper are available at:

<https://github.com/VesterlundCoder/quantum-representation>

The repository contains:

- **data/** — the full dataset of 4,608 observations (`scale_observations_v2.json`)
- **figures/** — all generated figures (PNG, 300 DPI)
- **manuscript/** — LaTeX source for this paper
- **code/** — experiment script and figure-generation scripts

All numbers and percentages in the manuscript are derived programmatically from the dataset. The experiment can be reproduced by running:

```
pip install westquant-core==0.2.0a3 westquant-qiskit==0.1.0a4 \
    westquant-pytket==0.1.0a2 westquant-pennylane==0.1.0a1 \
    westquant-bridges==0.1.0a2 westquant==0.1.0a2
python code/scale_experiment_v2.py
python code/generate_figures.py
```

Declaration of Competing Interests

The author declares no competing interests. This research received no external funding. The author has no financial or personal relationships with any of the organizations or products discussed in this work that could inappropriately influence the work presented.

AI Use Disclosure

Generative AI tools (Claude/Anthropic and ChatGPT/OpenAI) were used during the preparation of this manuscript for the following purposes: generating figures and visualizations, drafting and editing prose, formatting the LaTeX manuscript, and assisting with literature search and screening. The research idea, research questions, taxonomy design, formal mathematical framework, classification of all papers, interpretation of findings, and all scientific judgments are solely the work of the author. Every AI-generated figure, table, and text passage has been reviewed, verified, and corrected by the author. The author takes full responsibility for the accuracy and integrity of all content in this manuscript.